

CReSTIC

Rapport de stage

Stage de L2

CAPITANO Giuliano

Du 06/04/26 au 29/05/26

Tuteur de stage : M. PHILIPPOT

Tuteur enseignant : M. RABAT Cyril

Établissement : URCA - UFR SEN

**Diplôme : Licence 2 Informatique & CMI INFO -
2025 / 2026**

Table des matières

I.	INTRODUCTION.....	3
II.	ORGANISME D'ACCUEIL	3
I.	CRESTIC	3
II.	CONTEXTE DU STAGE	3
III.	ENVIRONNEMENT DE TRAVAIL & TECHNOLOGIES.....	4
III.	LITE3	4
IV.	ECOSYSTEME	4
V.	APPRENTISSAGE PAR RENFORCEMENT	4
VI.	ORDINATEUR	5
VII.	SIMULATION	6
VIII.	GIT & GITHUB	6
IV.	TACHES REALISEES	6
IX.	REALISATIONS	6
a.	<i>Saut en longueur</i>	6
b.	<i>Dire bonjour</i>	8
c.	<i>Equilibre sur pattes arrière</i>	10
X.	DEPLOIEMENT	11
d.	<i>Machine à états</i>	11
e.	<i>Simulation</i>	12
f.	<i>Réel</i>	13
g.	<i>Interface mobile</i>	13
V.	CONCLUSION	15
XI.	RESSENTI	15
XII.	AMELIORATIONS	15
XIII.	REMERCIEMENTS.....	15
VI.	RESSOURCES.....	15

I. Introduction

Dans le cadre de ma Licence Informatique et de mon DU CMI Informatique à l'Université de Reims Champagne-Ardenne, j'ai réalisé un stage de 2 mois du 06/03/2026 au 29/05/2026 au sein du CReSTIC. Ceci en est le rapport.

II. Organisme d'accueil

i. CReSTIC

Le Centre de Recherche en STIC (CReSTIC) est une unité de recherche de l'URCA. Ces recherches sont dans le domaine des sciences du numérique et est présent sur tous les sites de l'URCA (Reims, Troyes, Châlons-en-Champagne, Charleville-Mézières).

Le CReSTIC est constitué de 60 enseignants-chercheurs permanents et une quarantaine de personnels supplémentaire. On peut d'ailleurs y indiquer la présence de praticiens hospitaliers liées à certaines thématiques de recherche du CReSTIC.

Le laboratoire est séparé en 5 équipes de recherche, chacune couvrant des activités fondamentales telles que l'automatique, l'informatique, le traitement du signal et des données, l'apprentissage automatique et l'image. C'est une unité de recherche multidisciplinaire qui couvre tout autant le côté *hardware* (matériel) que le côté *software* (logiciel).

On peut donc énoncer les 5 équipes de recherche, ayant chacune sa spécialité :

- AI4M (Artificial Intelligence for Medecine)
- CO2S2CP (Commande, Observation, Optimisation et Sécurité des Systèmes Cyber-Physiques)
- Num&Soc (Numérique et société)
- TICA (Traitement d'Images et Commande Avancée)
- TS (Traitement du Signal)

Cette unité de recherche est un acteur majeur de la recherche et l'innovation en Grand Est et s'implique dans bon nombre de projets académiques nationaux et internationaux.

Il possède notamment des relations fortes avec des PME et grands groupes français et avec le SATT Nord qui facilite le déploiement des technologies et le transfert des connaissances entre le secteur académique et les entreprises privées.

ii. Contexte du stage

Le CReSTIC utilise principalement le robot quadrupède Lite3 comme moyen de communication pour mettre en avant l'université de Reims en général, dont le CReSTIC lui-même et l'EiSIne (École d'ingénieurs en Sciences Industrielles et Numérique).

Le Lite3 (produit par Deep Robotics) possède une télécommande ainsi qu'une application permettant de le contrôler via Wifi. Cette télécommande permet de le déplacer et de lui faire faire des « actions complexes » tels que de dire bonjour, sauter vers l'avant, sauter sur place ou encore faire du moonwalk.

Or la dernière mise à jour logicielle du Lite3 effectuée a endommagé sa possibilité de réaliser plusieurs « actions complexes », dont dire bonjour et sauter vers l'avant. Mon stage

consistait alors à implémenter de zéro ces mouvements ainsi que de les déployer sur le robot.

Il me sera plus tard demander d'implémenter au robot l'action de se tenir en équilibre sur les pattes arrière afin de voir les limites techniques, mécaniques et sensorielles du robot.

III. Environnement de travail & technologies

iii. Lite3

Le Lite3 est un robot quadrupède à multi-usages, c'est-à-dire que chacun peut l'adapter à ces besoins. Supportant un port de charge entre 2.5 kilogrammes et 5 kilogrammes, le Lite3 peut être utilisé par exemple pour de la livraison ou pour faire des figures acrobatiques. Il existe actuellement 4 versions du Lite3 : Basic, Venture, Pro, Lidar.

Les 3 premières versions sont simplement le Lite3 avec de plus en plus de puissance. La version Lidar possède les mêmes spécifications techniques que la version Pro mais avec un peu plus de masse (due au lidar sur son dos), une distance parcourable et un port de charge moins important. Le Lite3 est composé de plusieurs composants, dont un ordinateur embarqué et un microcontrôleur. L'ordinateur embarqué communique via ROS2 les informations au microcontrôleur qui lui, avec les bons drivers des moteurs et des actionneurs, va faire réaliser l'action au robot.

Le Lite3 du CReSTIC est la version Lidar, il possède donc le lidar sur le dos. Je dois alors faire encore plus attention lors de la création des mouvements afin de ne pas endommager le robot. En effet le modèle 3d mis à disposition par Deep Robotics est celui de la version sans lidar. L'IA entraîné via apprentissage par renforcement (RL) ne réagira pas de la même manière pour un Lite3 avec du poids supplémentaire.

De plus, l'environnement d'entraînement IA via RL ne traduit pas l'environnement réel, c'est la simulation 3d qui se charge de cela. Vous allez voir que cela possède une importance pour l'un des mouvements.

iv. Ecosystème

Deep Robotics possède un compte github et publie en « open-source » la majorité des programmes et technologies nécessaires à un utilisateur pour améliorer le Lite3. Ceci est très pratique car cela permet de rapidement mettre en place l'environnement de travail. Je me suis intégralement basé sur leurs ressources pour comprendre comment fonctionne le robot et comment le modifier. Deep Robotics a aussi un serveur Discord dédié qui permet aux utilisateurs de poser des questions directement aux développeurs.

v. Apprentissage par renforcement

L'apprentissage par renforcement (RL) est la manière la plus simple et directe de faire apprendre à un agent autonome à réaliser une tâche. Le RL est une branche de l'apprentissage automatique qui permet à l'agent d'apprendre des actions à partir d'expériences précédentes pour optimiser une récompense dans le temps.

On place alors l'agent dans un environnement dont il peut en récupérer des informations (hauteur, vitesse, inclinaison, etc). L'agent effectue alors une action sur l'environnement (forcément aléatoire au départ). L'action est interprétée par l'environnement est y attribue un score (la récompense) ainsi que l'état suivant. Ces 2

informations sont transmises à l'agent qui va alors s'adapter en tenter de maximiser sa récompense et/ou minimiser sa pénalité.

Dans un vrai scénario de RL, l'agent possède plusieurs fonctions de récompenses et plusieurs fonctions de pénalités. Il va alors tenter de maximiser la récompense finale avec la formule suivante :

Avec :

- $\mathbb{X} = \{x_1, \dots, x_{|\mathbb{X}|}\}$ l'ensemble des observations de récompenses,
- $\mathbb{Y} = \{y_1, \dots, y_{|\mathbb{Y}|}\}$ l'ensemble des observations de pénalités,
- $i \in \llbracket 1, \dots, |\mathbb{X}| \rrbracket$,
- $j \in \llbracket 1, \dots, |\mathbb{Y}| \rrbracket$,
- $\alpha_i \in \mathbb{R}_+^*$ le poids associé à la récompense x_i ,
- $\beta_j \in \mathbb{R}_+^*$ le poids associé à la pénalité y_j ,

On a :

$$R = \sum_{i=0}^{|\mathbb{X}|} (x_i \alpha_i) - \sum_{j=0}^{|\mathbb{Y}|} (y_j \beta_j)$$

Deep Robotics utilise l'apprentissage par renforcement via les outils développés par Nvidia IsaacSim et IsaacLab.

IsaacSim est un environnement de simulation pour robots permettant de réaliser de multiples tâches telles que la conception de robots, la simulation et le tests de ces mêmes robots contrôlés par IA. IsaacSim est basé sur Omniverse et offre un contrôle complet de toutes les étapes pour l'utilisation de robots avec IA, notamment avec le support natif de ROS.

De son côté, IsaacLab est un framework Python basé sur IsaacSim qui permet la création d'IA par apprentissage par renforcement selon l'environnement et les récompenses. Gymnasium est utilisé dans le script produit par Deep Robotics pour simplifier la gestion des environnements d'entraînements.

Par simplicité, Nvidia recommande d'installer IsaacLab et IsaacSim à l'intérieur d'un conteneur python nommé 'Conda' et d'analyser les résultats d'une IA générée avec IsaacLab avec 'Tensorboard' afin de suivre à l'aide de graphes l'avancée de l'IA, le nombre de récompenses, la valeur de chacune d'entre elles, etc.

Le résultat de l'entraînement par renforcement est nommé une « politique » (policy en anglais) et est stocké dans un fichier .ONNX qui contient les valeurs du réseau neuronal qui dictent les actions réalisées par le robot selon les valeurs qu'il capte.

J'ai dû notamment modifier les scripts de Deep Robotics car ils embarquaient des valeurs obsolètes qui provoquaient des erreurs lors de l'initialisation de la configuration de l'environnement d'apprentissage par renforcement.

vi. Ordinateur

Un GPU Nvidia compatible CUDA est nécessaire pour utiliser l'environnement IsaacLab et IsaacSim préconfiguré par Deep Robotics sur leur dépôt 'rl_training' pour le Lite3, or mon ordinateur portable possède un GPU Intel. Par chance, le CReSTIC possède un unique

ordinateur portable avec GPU Nvidia compatible CUDA, le GeForce RTX 2070 Portable qui est de la première génération de GPU Nvidia compatibles CUDA.

Afin de réaliser un entraînement par renforcement sur le Lite3 avec un ordinateur ne possédant pas de GPU Nvidia, j'aurais dû moi-même créer un environnement d'entraînement à l'aide de MuJoCo, mais cela prendrait plusieurs semaines à mettre en place. J'ai alors préféré prendre l'ordinateur du laboratoire malgré la lenteur de son GPU.

vii. Simulation

Pour vérifier le réseau neuronal généré par IsaacLab, Deep Robotics met aussi à disposition un environnement de simulation « réel » qui simule exactement le comportement du Lite3 selon sa masse, la force gravitationnelle, les vraies contraintes physiques, le terrain, etc. Le dépôt 'Lite3_rl_deploy' utilise MuJoCo ou PyBullet. On pourra par la suite voir que l'IA générée peut paraître stable dans IsaacSim mais totalement instable dans MuJoCo.

viii. Git & Github

Dans le but de garder traces des tâches réalisées durant mon stage et de mes avancées jours par jours et d'y apporter de la documentation en français, j'ai réalisé un fork des dépôts de Deep Robotics 'rl_training', 'Lite3_rl_deploy' et 'gamepad'. La documentation réalisée peut servir de base pour une autre personne souhaitant réaliser ses propres actions sur un Lite3 et voulant ensuite déployer cela.

J'ai ensuite créé une dépôt dédié à mon stage complet. Il contient les liens vers les dépôts cités plus haut ainsi que des vidéos montrant les échecs et réussites des mouvements que j'ai réalisés.

IV. Taches réalisées

ix. Réalisations

a. Saut en longueur

La première action que j'ai dû implémenter a été le saut vers l'avant. Dans l'idée, ce saut doit permettre au robot de décoller mais en priorisant l'avancée, la hauteur atteinte n'étant que secondaire.

Pour ce faire, il faut d'abord créer un nouveau fichier dans la liste des environnements disponibles pour IsaacLab qui est la configuration de l'environnement pour ce nouveau mouvement. On définit alors dedans l'ensemble des fonctions de récompenses et de pénalités ainsi que les poids associés.

Un second fichier, celui-ci étant la configuration de l'agent (le robot), prends le nom de l'expérience (nom du répertoire des logs), gamma, le coefficient d'entropie et le nombre maximum d'itérations pour la version X du mouvement.

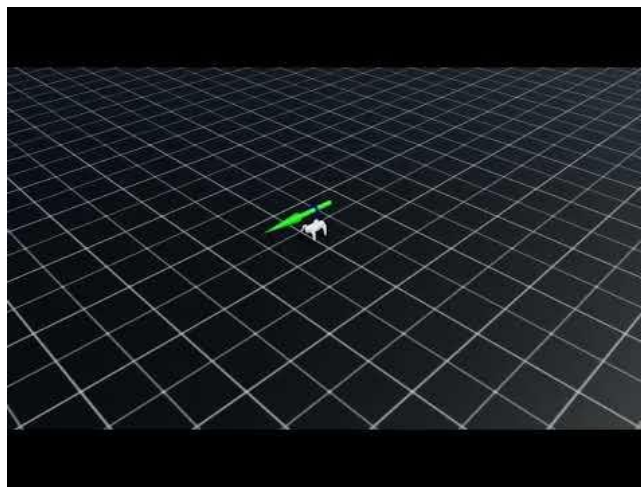
Pour finir, il faut ajouter dans le fichier __init__.py (qui est le constructeur des environnements Gymnasium) une entrée liant le fichier de configuration de l'environnement et le fichier de configuration de l'agent. On y indique le nom que l'on va entrer en lançant le script python avec la version associée. Par exemple l'argument '--task=Lite3-Flat-v0' lancera le mouvement 'Flat' associé à la version 0. Le script va alors charger le fichier de configuration de l'environnement, y chercher la fonction de la version 0 et en même temps le fichier de configuration de l'agent.

Il est également possible de prendre comme argument du script d'entraînement un checkpoint afin de reprendre l'entraînement depuis un réseau de neurones déjà entraîné. J'ai réalisé en tout 12 versions de l'environnement pour le saut en longueur. Les versions 0 à 4 essayaient de faire apprendre au robot à sauter vers l'avant avec comme unique expérience préalable le mouvement 'Flat' qui fait que le robot sait se tenir debout et est stable sur un sol plat. Or un saut est un mouvement complexe et il est très peu probable que le chien le découvre aléatoirement.

J'ai alors décidé de réaliser plusieurs versions de l'environnement qui prennent chacune comme checkpoint la version précédente afin que le chien apprenne petit à petit des « étapes » du saut jusqu'à ce qu'il puisse réaliser toutes les étapes d'une traite. La version 5 prends comme checkpoint, comme avant, le mouvement 'Flat'. La version 6 prends comme checkpoint la version 5, et chacune des versions suivante continue comme cela jusqu'à la version 12. Le chien va par exemple apprendre dans un premier temps à se pencher un peu vers l'avant, ensuite à avoir de la coordination entre les pattes, puis apprendre à décoller. Cela semble simple sur papier mais devient vite complexe si l'agent trouve des solutions qui maximisent ces récompenses tout en s'éloignant du résultat voulu : c'est un problème d'alignement de l'IA.

J'ai passé 1 semaine et demie à faire l'entraînement de l'IA pour le saut en longueur avec 12 versions « concluantes » de configurations d'environnements et une vingtaine en prenant en compte les versions « concluantes » et « non concluantes ». Ceci représente à peu près 30 heures de pur entraînement de l'IA. J'ai malheureusement dû arrêter au bout de la semaine et demie à cause d'une chose toute bête : le robot saute mais atterrit sur les genoux, ce qui va l'endommager si déployer sur le vrai robot.

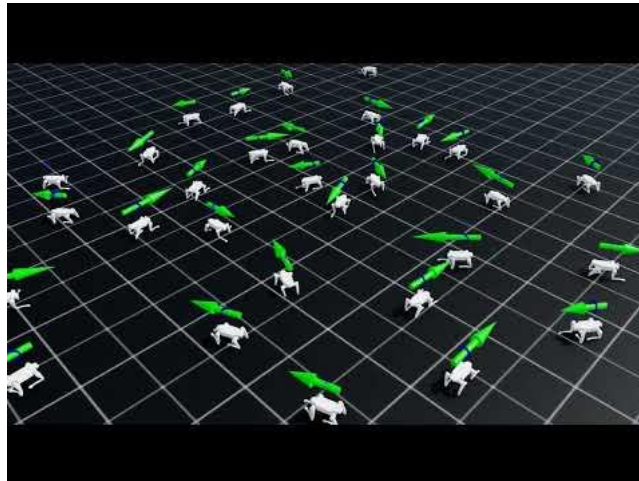
C'est ici un problème d'alignement : en effet, j'avais stipulé plusieurs dans la version 9 que le robot gagnait plus de récompenses lorsque ces pieds ne touchaient pas le sol, pensant naïvement que cela le forcerait à sauter. Le robot avait alors trouvé une action qui maximisait les récompenses mais qui n'était pas un saut : tout simplement replier ces jambes afin que ces pieds ne soient pas en contact avec le sol. Un saut requiert d'abord une phase « debout », les pieds au sol afin de prendre une impulsion pour se propulser en l'air. Or cette phase est déjà une perte potentielle de points pour l'agent : le robot ne le fait donc pas et se recroqueville immédiatement afin de gagner un maximum de points.



Problème d'alignement pour le « saut » qui cause un repliement des jambes dans IsaacSim

La version suivante a marqué l'ajout d'une pénalité supplémentaire : les jambes repliées en étant « proche » du sol, le robot a enfin appris à sauter. Malheureusement, rien ne lui stipulait de ne pas le faire dans les airs. Il sautait alors et repliait ces jambes dans les airs, puis atterrissait aussi sur les genoux pour gagner toujours plus de points. Pour régler ce problème d'alignement, il aurait fallu réaliser une IA bien plus complexe qui embarquerait une machine à état lui disant que pour l'état X, les fonctions Y s'y appliquent, mais cela est bien plus complexe et chronophage. J'ai alors décidé d'abandonner la réalisation de ce mouvement afin de me concentrer sur le suivant : « dire bonjour ».

Dans certains cas, si le robot est trop fortement pénalisé, il est plus sûr pour lui de ne rien faire afin de ne pas perdre des points. C'est ce qui se passe dans cette vidéo :



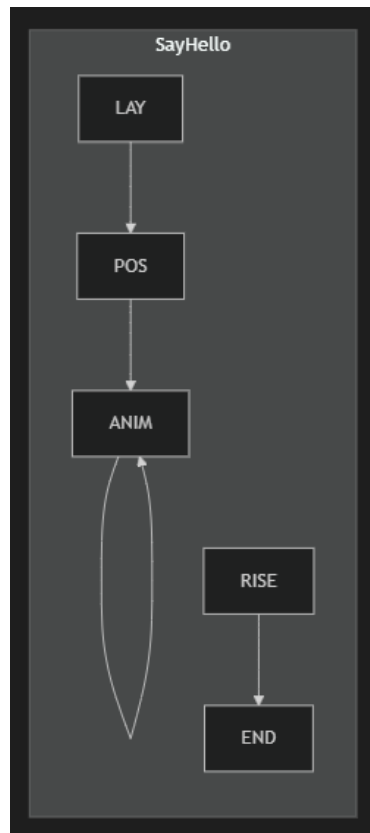
Problème d'alignement pour le « saut » qui cause une immobilisation du robot dans IsaacSim

L'entraînement par RL n'est finalement qu'un jeu du chat et de la souris entre le développeur et l'IA : je souhaite faire apprendre tel mouvement à l'agent via des fonctions de récompenses et de punitions. Si l'environnement et les fonctions de récompenses / punitions ne sont pas assez précises, l'agent trouvera une action produisant un maximum de points mais qui est différente de l'action souhaitée. Dans ce cas, l'action souhaitée n'est qu'un maximum local, pas le maximum absolu de la fonction représentant le nombre de points obtenables par l'IA. Je dois ensuite modifier le paramétrage afin de « patcher » ce comportement non-voulu, répéter ce processus le nombre de fois nécessaires jusqu'à ce que l'action apprise par l'IA soit correcte.

b. Dire bonjour

« Dire bonjour » est le second mouvement que j'ai dû implémenter, il doit permettre au chien de lever son buste, de se maintenir sur une patte en la déplaçant en dessous de son corps et de lever la seconde patte afin de réaliser un mouvement de l'épaule et du coude pour saluer le public. Ce geste est réalisé à l'infini, il continue tant que l'utilisateur n'appuie pas sur le bouton pour l'arrêter. Etant donné que ce mouvement ne sera réalisé qu'à des fins de communication et demande un contrôle précis et complet sur chaque joint du robot, je l'ai réalisé « à la main » en C++.

« Dire bonjour » est une action complexe qui réalise des mouvements en fonction du temps, sans prendre en compte son environnement. Il peut alors être séparé en plusieurs sous états qui décrivent mieux l'enchaînement des actions. J'ai décidé de le séparer en 5 étapes bien distinctes.



Graphe des sous-états de « dire bonjour »

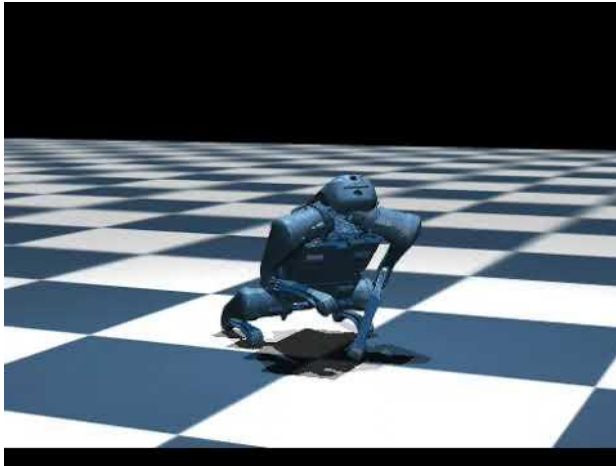
Le sous-état « LAY » couche le robot sur le ventre afin de le mettre en « position neutre ». Si le robot se couche, sa position précédente n'influence pas sur la position qu'il aura après sa phase « LAY », cela permet de commencer l'état « Dire bonjour » toujours dans la même position.

Le sous-état « POS » veut dire « Position » et met le robot en position juste avant de faire l'animation. Il va relever son buste, mettre sa patte sous son corps pour gagner en stabilité. Il passe alors en mode « ANIM ».

« ANIM » est le mouvement d'animation du robot. Pour le faire, j'utilise 2 sous-sous-états : « LIFT » et « MOVE ». « LIFT » monte la patte du robot et « MOVE » utilise la fonction sinus afin de réaliser un mouvement cyclique et infini sur le coude et l'épaule.

« RISE » est déclenché à la demande de l'utilisateur lorsqu'il souhaite que le robot se relève. Pour la même raison que précédemment, cet état est une sécurité pour que le robot sorte de « Dire bonjour » en toute sécurité. Pour finir, « END » est un état passager qui fait simplement sortir de « Dire bonjour ».

Voici les vidéos du rendu final de « Dire bonjour » déployé sur simulateur et en réel :



« dire bonjour » dans simulation MuJoCo



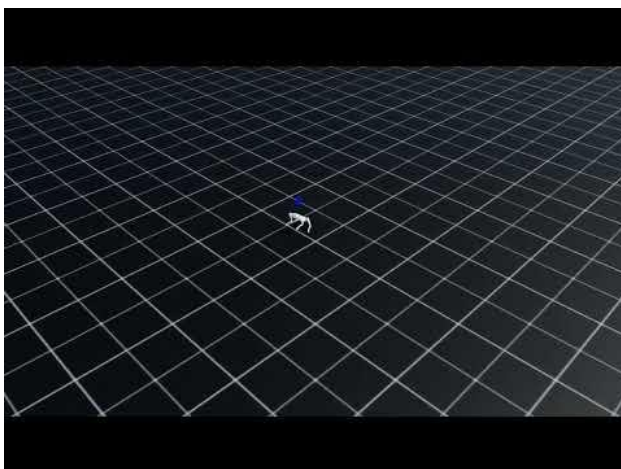
« dire bonjour » en réel

Il est à noter qu'étant donné que j'ai réalisé le mouvement sur le modèle 3d ne possédant pas le Lidar, le buste du vrai robot est un peu plus bas à cause de la masse supplémentaire.

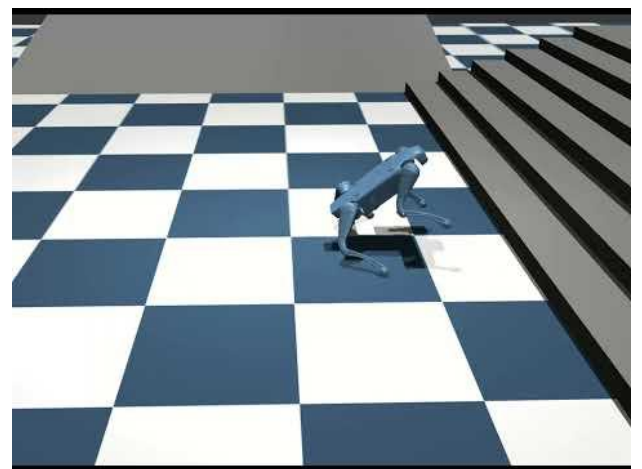
c. Equilibre sur pattes arrière

L'action d'« équilibre » sur les pattes arrière pour le Lite3 que j'ai réalisé est une action que mon tuteur de stage m'a proposé 3 semaines avant la fin du stage, courant Mai. Elle avait pour but de montrer l'agilité du Lite3 et toutes les possibilités physiques de déplacement du robot. Ayant eu l'expérience avec le « saut », j'ai fait uniquement 4 versions d'environnements avec les récompenses et pénalités nécessaires. Comme avant, la V1 prend comme checkpoint le fait que le robot sait être debout et stable, la V2 prend la V1, la V3 prend la V2 et la V4 prend la V3.

Après ces 4 versions, le robot savait se tenir totalement en équilibre dans l'environnement IsaacSim. J'ai malheureusement eu une surprise lors du déploiement en simulation : l'agent n'est pas du tout stable dans la simulation MuJoCo qui simule le monde réel. Pour preuve, voici 2 vidéos : la première étant la simulation IA d'IsaacSim et la seconde étant la simulation MuJoCo qui utilisent pourtant l'exacte même politique.



« équilibre » dans IsaacSim



« équilibre » non stable dans simulation MuJoCo

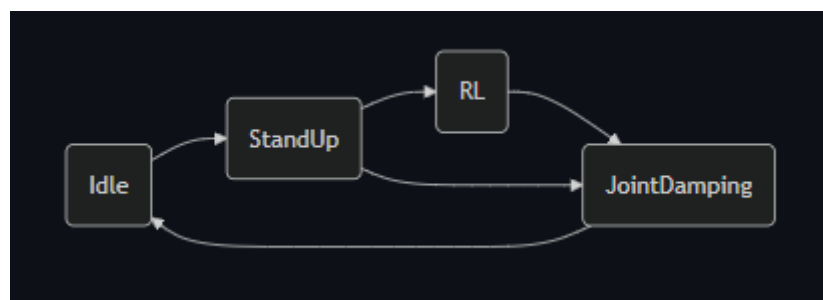
Si cette politique était déployée sur le robot, cela le casserait simplement. Vous pouvez remarquer à la fin de la vidéo de la simulation, le robot a détecté un comportement dangereux et est entré en sécurité.

x. Déploiement

d. Machine à états

Pour déployer les mouvements créés par apprentissage par renforcement, que ce soit en simulation ou en réel, j'ai utilisé le dépôt 'Lite3_rl_deploy' qui permet d'ajouter des comportements au robot sur simulation ou en réel. Pour ajouter des actions et simplifier le passage de l'une à l'autre, Deep Robotics utilise une « machine à états ». Cette machine à état indique dans quel état est le robot et dans quel état doit-il entrer ensuite selon la touche appuyée.

Voici la machine à état initiale :



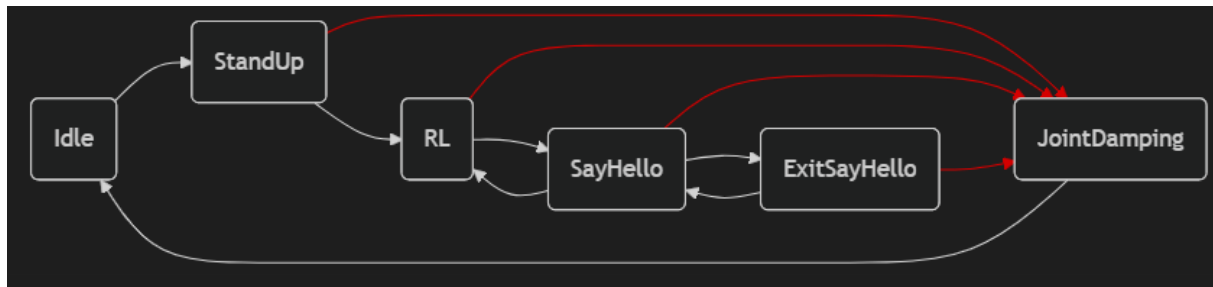
Machine à états initiale

Les différents états ont chacun un rôle bien précis et chaque état définit quel sera l'état suivant selon les entrées de l'utilisateur. « Idle » est l'état initial du robot. Le robot ne fait rien et attend que l'utilisateur lance « StandUp ». « StandUp » lève simplement le robot et attend que l'utilisateur lance « RL ».

« RL » (pour Reinforcement Learning) est l'état dans lequel cohabite plusieurs politiques IA. Ces politiques sont celles entraînées précédemment avec IsaacLab et IsaacSim. Chacune d'entre elles définissent leur prochain état à l'appui de telle touche (comme les états). Par exemple, on peut avoir une politique « marcher » et une politique « courir ». Si en étant dans « marcher » j'appuie sur 'c', j'entre dans « courir », et dans « courir » je peux appuyer sur 'm' pour retourner en « marcher ». Le dépôt contient une unique politique qui fait marcher le robot sur un terrain plat et j'ai décidé de la garder car elle fonctionne assez bien.

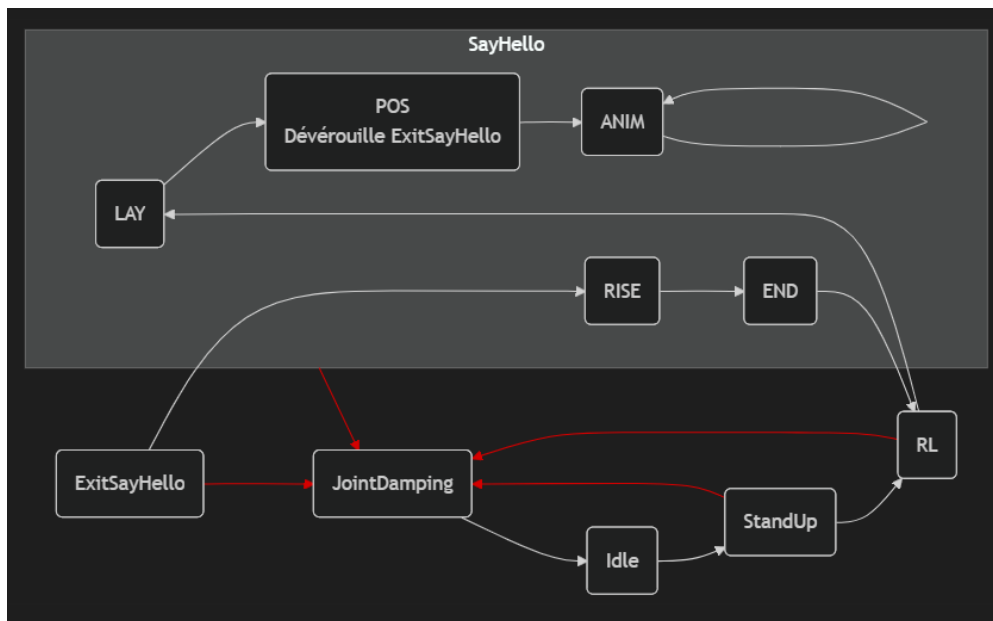
L'état « JointDamping » est l'état de mise en sécurité qui se déclenche lorsque le robot réalise une action pouvant endommager le matériel. Il amortit alors les joints du robot en toute sécurité. C'est par exemple ce qui s'est produit dans la vidéo de l'« équilibre » dans la simulation : le robot faisait des mouvements trop brusques et est passé en mode sécurité.

Pour rajouter des politiques IA, il suffit d'ajouter dans l'état « RL » la politique et comment y entrer et sortir. A contrario, pour ajouter une action faite « à la main » comme « dire bonjour » il faut créer un tout nouvel état. J'ai donc ajouté l'état « SayHello » et l'état « ExitSayHello » qui sert à sortir du « dire bonjour ». Voici le graphe de la machine à états avec ces 2 états en plus :



Machine à états avec états pour « dire bonjour »

On remarque des relations complexes entre « RL », « SayHello » et « ExitSayHello » car, comme vu précédemment, « SayHello » est composé de plusieurs sous-états. Voici donc le graphe « complet » de la machine à états :



Machine à états complète intégrant « dire bonjour »

« RL » envoie vers « LAY », puis vers « POS » et « ANIM ». Lorsque « POS » est enclenché, cela débloquent « ExitSayHello » qui fait se lever en sécurité le robot et le fait sortir avant de le faire revenir en « RL ». La flèche rouge sortant du sous-graphe indique que chacun des sous-états de « SayHello » peut déclencher la mise en sécurité.

Pour ajouter de nouveaux états d'actions « faites à la main », il est possible d'ajouter avec l'état « MonAction » un état « ExitMonAction » si cette action est faite à l'infini pour en sortir en toute sécurité. Si c'est le cas, « MonAction » aura sûrement besoin de sous-états (comme « SayHello »). Si l'action « MonAction » se fait en un temps défini et s'arrête ensuite, il suffit de relier « RL » à « MonAction » puis faire l'inverse pour en sortir. Attention, tout état et sous-état doit pouvoir passer en mode sécurité, d'où le nombre de flèches rouges dans le graphe.

e. Simulation

Comme indiqué plus tôt, le déploiement en simulation sert à vérifier que l'IA ou le mouvement créé à la main est valide et peut être ensuite déployé sur le vrai robot. Etant donné qu'il n'y a pas de robot en réel pour gérer les moteurs, la simulation MuJoCo gère toutes les valeurs que le vrai robot connaîtrait (torque, valeurs des joints, etc).

Etant donné que tout est exécuté sur un ordinateur, le seul moyen de contrôler le robot est par le clavier. On peut donc compiler et lancer l'application 'rl_deploy' afin d'envoyer à l'ip 127.0.0.1 :20001 les données du clavier, et la simulation qui écoute le port 20001 va faire déplacer le robot.

f. Réel

Le déploiement en réel est la dernière étape qui sert de vérification finale. Pour ce faire, il faut se connecter en SSH sur l'ordinateur embarqué du Lite3, compiler et lancer l'application 'rl_deploy'. Elle peut être configuré pour contrôler le robot via le clavier ou en utilisant la manette fournie avec le robot, le Retroid Gamepad.

Cette manette est un téléphone sous Android avec des joysticks et gâchettes. On peut donc lancer l'application associée sur le téléphone, y spécifier l'ip et le port pour communiquer en UDP avec l'ordinateur embarqué du Lite3. Je peux donc décider avec quel terminal d'entrées je peux contrôler le robot et modifier une unique ligne du programme qui s'exécute sur l'ordinateur embarqué pour paramétrer le contrôle du robot via clavier ou manette.

g. Interface mobile

Sur la manette fournie avec le robot est installée une application très complète, différente de la précédente, qui contient énormément de fonctionnalités pour contrôler le Lite3. Avec celle-ci, il est possible de faire marcher, courir, sauter, dire bonjour, un moonwalk le robot chien. Or, c'est cette même application qui, lorsque l'action « sauter » ou « dire bonjour » est enclenchée, ne fait rien faire au robot. Je souhaitais donc la modifier afin de supprimer l'ancien « dire bonjour » et le remplacer par celui que j'ai créé. De plus, il serait très simple de rajouter des actions aux robots par ce biais, car il suffirait de rajouter un bouton dans l'application et d'envoyer les données via UDP à l'ordinateur embarqué.

Mais après avoir posé la question à l'un des développeurs chez Deep Robotics, il m'a été indiqué que cette application est propriétaire et qu'elle le restera. Il m'est donc impossible de modifier le code source de cette application. Décompiler le fichier apk ne fonctionne pas non plus car cela donne de l'assembleur, impossible donc à travailler avec. La seule solution qui est pour le moment « viable » est ce qui est indiqué dans la partie précédente, l'application portable du dépôt 'gamepad'.



Application « tout-en-un » propriétaire



Page des actions dans l'application « tout-en-un » propriétaire

L'application du dépôt 'gamepad' fonctionne très bien pour contrôler le robot mais il n'y a pas d'affichage intuitif. L'utilisateur qui déploie et exécute la machine à états sur le robot doit appuyer sur une touche pour faire se lever le robot et une autre pour le faire basculer en mode « RL » pour ensuite pouvoir faire marcher le chien. L'application affiche ce que voit la caméra du chien mais aucun élément interactif sur l'écran n'est présent.



Application du dépôt « gamepad » qui n'affiche que la caméra du robot

Sur la semaine et demie restante de stage, je vais devoir créer une application mobile avec des boutons permettant d'améliorer l'expérience utilisateur. L'utilisation des boutons « physiques » du retroid ne sera nécessaire que pour le déplacement du robot, tout le reste (se lever et dire bonjour) sera réalisé en appuyant sur un bouton sur l'écran. J'ai toutefois des réserves sur la possibilité de faire cela en 1 semaine et demie car je n'ai pas non plus le code source de l'application 'gamepad'. Sur le github de Deep Robotics n'est présent que le fichier apk de cette application, pas le code. Je vais devoir demander le code source aux développeurs de Deep Robotics et, à défaut, comprendre comment et de quelle manière l'application récupère le flux vidéo de la caméra du robot par exemple.

Etant donné que je n'ai pas accès au code source de l'application tout-en-un de Deep Robotics, toutes les actions qui y sont présentes doivent être réimplémentées de zéro : moonwalk, courir, déplacement sur sol non plat, etc. Cela représente une grosse charge de travail et peut prendre beaucoup de temps. Ces actions devront être recrées soit par IA soit « à la main » comme mon « dire bonjour ».

Il me sera aussi nécessaire de compiler l'application 'rl_deploy' sur l'ordinateur embarqué et de la lancer au démarrage du robot via systemd. Il sera tout de même nécessaire de modifier le code et de recompiler lorsque de futures actions seront implémentées.

V. Conclusion

xi. Ressenti

Durant ce stage, j'ai dû faire preuve de curiosité, de persévérance et de beaucoup de recherches sur Internet afin de réaliser les tâches qui m'étaient imposées. En effet, les missions qui m'ont été confiées n'avaient aucune indication sur comment les réaliser, il était à moi de prendre l'initiative de rechercher les solutions existantes ou même de les créer moi-même.

Ce stage a été très formateur car j'ai dû chercher des moyens détournés de réaliser les actions demandées. Je suis tout de même frustré de n'avoir pu déployer qu'une seule action sur les trois demandées, et plus particulièrement pour l'action d'« équilibre » sur les pattes arrière car IsaacLab a produit une politique d'apprentissage par renforcement fonctionnelle mais la simulation ne l'interprète pas de la même manière, ce qui rend impossible son déploiement. Je pense que l'environnement d'IsaacSim, ne simulant pas aussi précisément que MuJoCo le monde réel, a laissé l'IA faire des micromouvements qui ne changent pas son équilibre mais qui ont un impact fort sur les mouvements dans MuJoCo. Je ne sais pas comment régler ce problème.

xii. Améliorations

Un grand axe d'améliorations pour poursuivre ce travail que j'ai commencé est la réalisation de tous les autres mouvements, notamment le « saut » qui devra être réalisé « à la main » et régler le problème de l'IA pour l'« équilibre » sur les pattes arrière et le développement d'une application mobile plus poussée et modulable afin de pouvoir rajouter facilement de nouvelles fonctionnalités.

xiii. Remerciements

Je souhaite grandement remercier mon tuteur de stage M. PHILLIPOT et le CReSTIC plus globalement pour m'avoir donné l'opportunité de faire mon stage au sein de ce laboratoire sur un sujet que j'apprécie. Je remercie aussi Mme. BISSEMO Christvie, ingénieure roboticienne chez ERM Automatisme pour ces conseils.

VI. Ressources

- Site du CReSTIC : <https://www.crestic.univ-reims.fr/fr/accueil>
- Site de Deep Robotics : <https://www.deeprobotics.cn/en/>
- Github de Deep Robotics : <https://github.com/DeepRoboticsLab>
- Apprentissage par renforcement : <https://www.cnil.fr/fr/definition/apprentissage-par-renforcement>
- IsaacSim : <https://developer.nvidia.com/isaac/sim>
- IsaacLab : <https://developer.nvidia.com/isaac/lab>
- Dépôt Github de mon stage : https://github.com/Rav3nPho3nix/Stage_L2
- Github contenant les fork des dépôts de Deep Robotics : <https://github.com/Rav3nPho3nix>
- MuJoCo : <https://mujoco.org/>
- PyBullet : <https://pybullet.org/>
- Gymnasium : <https://gymnasium.farama.org/index.html>